Miłosz Kutyła

Application Security — laboratories
Topic #5: Custom web application development

March 16, 2026

Contents

1. Introduction . . . 1
2. System description . . . 2
2.1. Purpose . . . 2
2.2. Security assumptions . . . 2
3. Component requirements . . . 2
3.1. Functional requirements . . . 2
3.2. Non-functional requirements . . . 2
4. Component architecture . . . 3
5. Database structure . . . 4
5.1. Users table . . . 4
5.2. Sessions table . . . 4
5.3. Posts table . . . 5
5.4. Comments table . . . 5
5.5. Ratings table . . . 5
5.6. Admin Action table (Audit Log) . . . 6
6. Database relations . . . 7
7. UML sequence diagrams . . . 8
7.1. Creating posts with attached files . . . 8
7.2. Adding comments . . . 9
7.3. Adding ratings . . . 10
7.4. Editing content . . . 11
7.5. Keyword post search . . . 12
8. Implementation . . . 13
8.1. Unauthenticated user view . . . 13
8.2. Authenticated user view . . . 15
8.3. Admin user view . . . 19
8.4. Details . . . 21
9. Conclusions . . . 22

1. Introduction

The goal of this lab was to extend the authentication system, completed as part of a previous laboratory exercise, into a small content-based web service and to apply practical application security mechanisms in a realistic scenario. The website used for creating the UML diagrams was `https://mermaid.live/`.

Note: In order to keep this report concise, I neglect describing the previously created elements of the authentication module, unless necessary.

2. System description

2.1. Purpose

The system I decided to go for is a microblog. The system should provide a platform for sharing short-form posts while enforcing strict role-based access control (RBAC) and security measures to protect user-generated content. I decided to go with the suggested actors. The system is designed around three distinct ones:

- Guest users (unauthenticated): can browse public posts and utilize keyword-based search functionality.
- Registered users (authenticated): can create new posts (including file uploads), interact via comments or ratings, and manage (delete) only their own content.
- Administrators (authenticated): possess full moderation privileges, including the ability to delete or manage any content (post, comment, user account).

2.2. Security assumptions

The following security assumptions are made for the module:

- The application operates over an encrypted channel (HTTPS) to prevent eavesdropping during data transmission.
- The database is not directly accessible from the public network.
- Server-side validation is the primary source of truth; client-side validation is only for User Experience (UX).
- CSRF tokens are enforced for all sensitive endpoints (using POST, PUT, or DELETE methods).
- Rate limiting is implemented on all endpoints to prevent brute-force attacks and abuse.
- Sessions are securely managed server-side with cryptographically secure session identifiers.

3. Component requirements

3.1. Functional requirements

This module has the following functional requirements:

1. Public feed: unauthenticated visitors can view a read-only stream of posts.
2. Post creation: authenticated users can upload text content and files (images/PDFs).
3. Interactions: authenticated users can comment on and rate posts.
4. Self-management: authenticated users can delete their own posts or comments.
5. Global moderation: administrators can edit or delete any post, comment, or user account.
6. Search: keyword-based search functionality accessible to all actors.

3.2. Non-functional requirements

The registration module has the following non-functional requirements:

1. RBAC: Every sensitive endpoint must verify the actor's permissions before execution.
2. Secure file handling: uploaded files must be stored in a non-executable directory, renamed to prevent path traversal, and scanned for malicious content.
3. Injection protection: all user-generated content (posts, comments, search queries) must be sanitized and handled using parameterized queries to prevent SQLi and XSS.
4. Rate limiting: content creation endpoints are rate-limited to prevent automated spamming.

4. Component architecture

The system follows a client-server architecture, communicating via a RESTful API:

- Frontend: implemented in React.js. It handles the presentation layer, form rendering, provide immediate local form validation, and display appropriate messages based on backend's responses.
- Backend: implemented in Python with Django. This layer exposes API endpoints, handles business logic, server-side validations, password hashing, token generation, user creation, and account activation.
- Database: relational SQLite3 database storing user identities and tokens.
- File storage: an isolated, non-executable server directory for user-uploaded media.
- SMTP server: PUT SMTP server `poczta.student.put.poznan.pl` using implicit SSL/TLS over port 465; it is used for sending activation emails.
- Google reCAPTCHA: frontend collects the reCAPTCHA token, and the backend verifies it with Google's reCAPTCHA API before processing the request.
- Deployment: the built frontend is served via nginx, which also serves as a reverse proxy for the Django backend. HTTPS is enforced for all external traffic and terminated at nginx. This termination point is chosen because the entire stack is hosted on a single server, meaning that internal communication occurs over the local loopback interface. Note: this choice was made to allow inspecting the internal HTTP traffic for demonstration and debugging, accepting the risk of sniffing the unencrypted internal traffic.
  The diagram of the designed system architecture is shown in Figure 1.

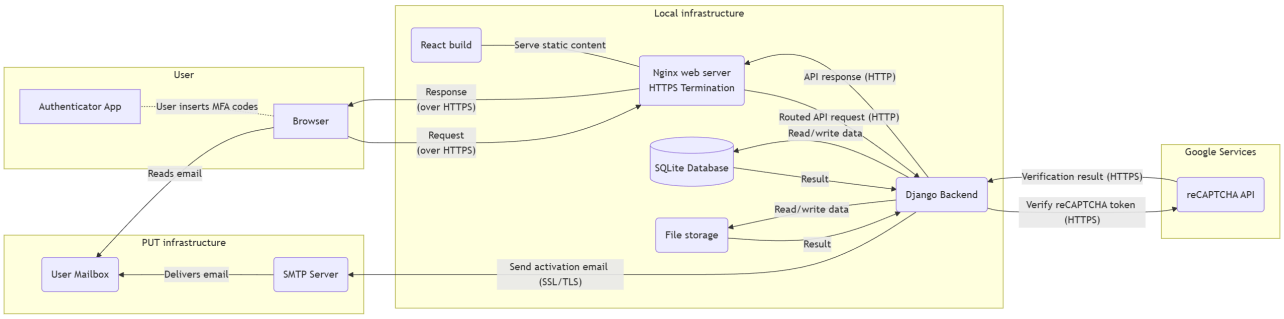


Figure 1: System architecture diagram

5. Database structure

5.1. Users table

The system utilizes a relational database to store user identities. The schema for the Users table (used to store user identities and tokens) is presented in Table 1.

Table 1: Structure of the Users table

Field name	Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Internal unique identifier.
`email`	Varchar(254)	Unique, Not Null	User's email, primary user identifier.
`password`	Varchar(128)	Not Null	Password string: algorithm, salt, hash.
`username`	Varchar(30)	Unique, Not Null	Public display name.
`date_joined`	DateTime	Not Null	Timestamp of registration.
`last_login`	DateTime	Nullable	Timestamp of last login.
`is_active`	Boolean	Default: False	Determines if the user can log in.
`is_staff`	Boolean	Default: False	Admin/moderator flag (RBAC role).
`is_superuser`	Boolean	Default: False	Django superuser flag.
`action_token_hash`	Varchar(64)	Unique, Nullable	Action token SHA256 hash.
`action_token_type`	Varchar(32)	Nullable	Action token type: email verification, password reset (...)
`token_token_expiry`	DateTime	Nullable	Timestamp when the token becomes invalid.
`mfa_secret`	Varchar(64)	Nullable	MFA secret.
`mfa_enabled`	Boolean	Default: False	Flag indicating whether user has MFA set up.
`mfa_backup_codes`	List	Nullable	List of backup codes' hashes.
`disabled_by_id`	Integer	FK (User.id), Nullable	Admin who disabled this account.
`disable_reason`	Text	Nullable	Reason for account disabling.
`disabled_at`	DateTime	Nullable	Timestamp when account was disabled.

5.2. Sessions table

The schema for the Sessions table (used to store active sessions) is presented in Table 2.

Table 2: Structure of the Sessions table

Field name	Type	Constraints	Description
`session_key`	Varchar(40)	PK, Not Null	Unique session identifier.
`session_data`	Text	Not Null	Serialized data associated with the session.
`expire_date`	Datetime	Not Null, Indexed	Expiry date of the session.

5.3. Posts table

The schema for the Posts table (used to store the blog entries) is presented in Table 3.

Table 3: Structure of the Posts table

Field name	Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Internal unique identifier.
`author_id`	Integer	FK (User.id), Not Null	Link to the content creator.
`title`	Varchar(64)	Not Null	Post title.
`content`	Text	Not Null	Sanitized user-generated text content.
`file_path`	Varchar(255)	Nullable	Path to the uploaded media file.
`created_at`	DateTime	Not Null	Timestamp of post creation.
`last_edited_at`	DateTime	Nullable	Timestamp of the last update; NULL if never edited.
`is_hidden`	Boolean	Default: False	Post's visibility flag.
`edited_by_id`	Integer	FK (User.id), Nullable	User who last edited (if admin).
`edit_reason`	Text	Nullable	Admin's reason for editing.

5.4. Comments table

The schema for the Comments table (used to store comments added under a post) is presented in Table 4.

Table 4: Structure of the Comments table

Field name	Type	Constraints	Description
`id`	Integer	PK, Auto Increment	Unique comment identifier.
`post_id`	Integer	FK (Post.id), Not Null	Link to the parent post.
`author_id`	Integer	FK (User.id), Not Null	Link to the commenter.
`text`	Text	Not Null	Sanitized comment text.
`created_at`	DateTime	Not Null	Timestamp of comment creation.
`last_edited_at`	DateTime	Nullable	Timestamp of last edit.
`is_hidden`	Boolean	Default: False	Comment's visibility flag.
`edited_by_id`	Integer	FK (User.id), Nullable	User who last edited (if admin).
`edit_reason`	Text	Nullable	Admin's reason for editing.

5.5. Ratings table

The schema for the Ratings table (used to store users' post ratings) is presented in Table 5.

Table 5: Structure of the Ratings table

Field name	Type	Constraints	Description
`user_id`	Integer	FK (User.id), Not Null	User who provided the rating.
`post_id`	Integer	FK (Post.id), Not Null	The post being rated.
`score`	Integer	Not Null, 1–5	Numerical value of the rating.
Composite PK			(`user_id`, `post_id`)

5.6. Admin Action table (Audit Log)

The schema for the Admin Action table (used to log all admin moderation actions) is presented in Table 6. This table provides an audit trail that survives even after content is hard-deleted.

Table 6: Structure of the AdminAction table

Field name	**Type**	**Constraints**	**Description**
`id`	Integer	PK, Auto Increment	Unique action identifier.
`action_type`	Varchar(20)	Not Null	Type: DELETE_POST, DELETE_COMMENT, DELETE_USER, DISABLE_USER, EDIT_POST, EDIT_COMMENT.
`admin_id`	Integer	FK (User.id), Not Null	Admin who performed the action.
`target_user`	Varchar(30)	Not Null	Username of the affected user.
`target_content_id`	Integer	Nullable	ID of affected post/comment/user (if applicable).
`target_content_type`	Varchar(20)	Nullable	Either 'post' or 'comment'.
`reason`	Text	Not Null	Admin's explanation for the action.
`content_snapshot`	Text	Nullable	Title/text of deleted content.
`created_at`	DateTime	Not Null	Timestamp when action was performed.

6. Database relations

I utilize a relational database structure where the Users table serves as the central entity for identity and authorization. The system's data integrity is maintained through the use of Foreign Keys (FK) and composite Primary Keys (PK) that link content and interactions directly to verified user identities. The diagram, shown in Figure 2, illustrates the following relational logic:

- Both the Posts and Comments tables utilize an `author_id` foreign key linked to the Users table to enforce RBAC during editing or deletion. A single user can author multiple posts and write multiple comments across the platform, including adding multiple comments under a single post.
- The Comments table is linked to the Posts table via a `post_id` foreign key, ensuring that all interactions are correctly associated with specific content.
- The Ratings table employs a composite primary key consisting of `user_id` and `post_id`. This prevents a single user from submitting multiple ratings for the same post.
- All primary content tables (Users, Posts, and Comments) include a boolean flag (`is_disabled` or `is_hidden`) to allow for administrative moderation without permanent data loss.
- AdminActions table utilizes an `admin_id` foreign key linked to the Users table to ensure auditability.
- The Sessions table remains decoupled from specific user records at the schema level. Authentication is maintained by storing the encrypted user identifier within the serialized session data, allowing the system to verify identities without a direct foreign key relationship.

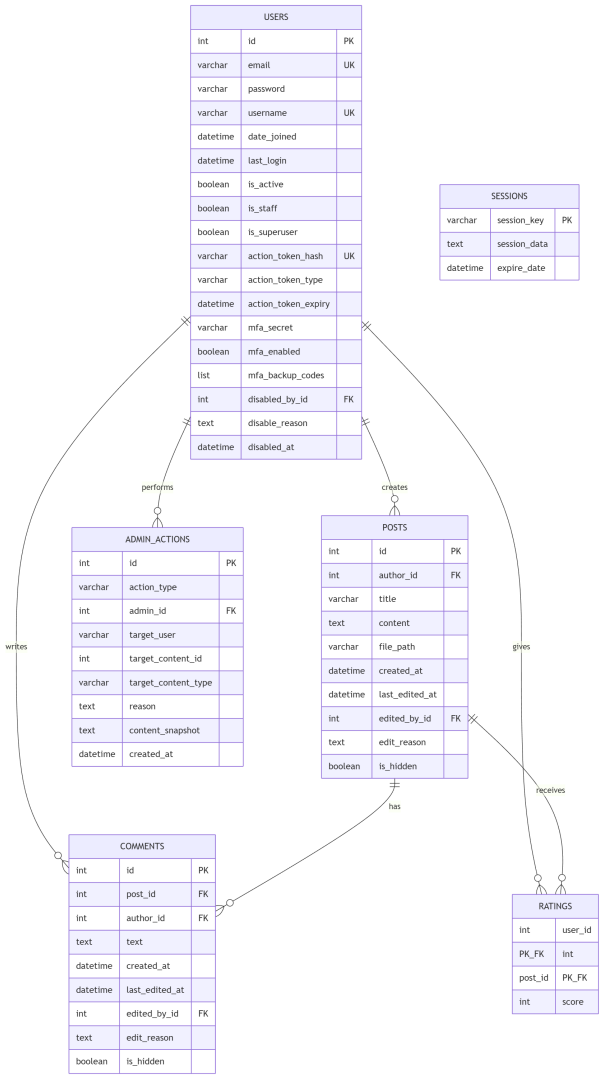


Figure 2: Diagram illustrating the table relations

7. UML sequence diagrams

7.1. Creating posts with attached files

The UML sequence diagram for creating posts (with file attachement) is shown in Figure 3.

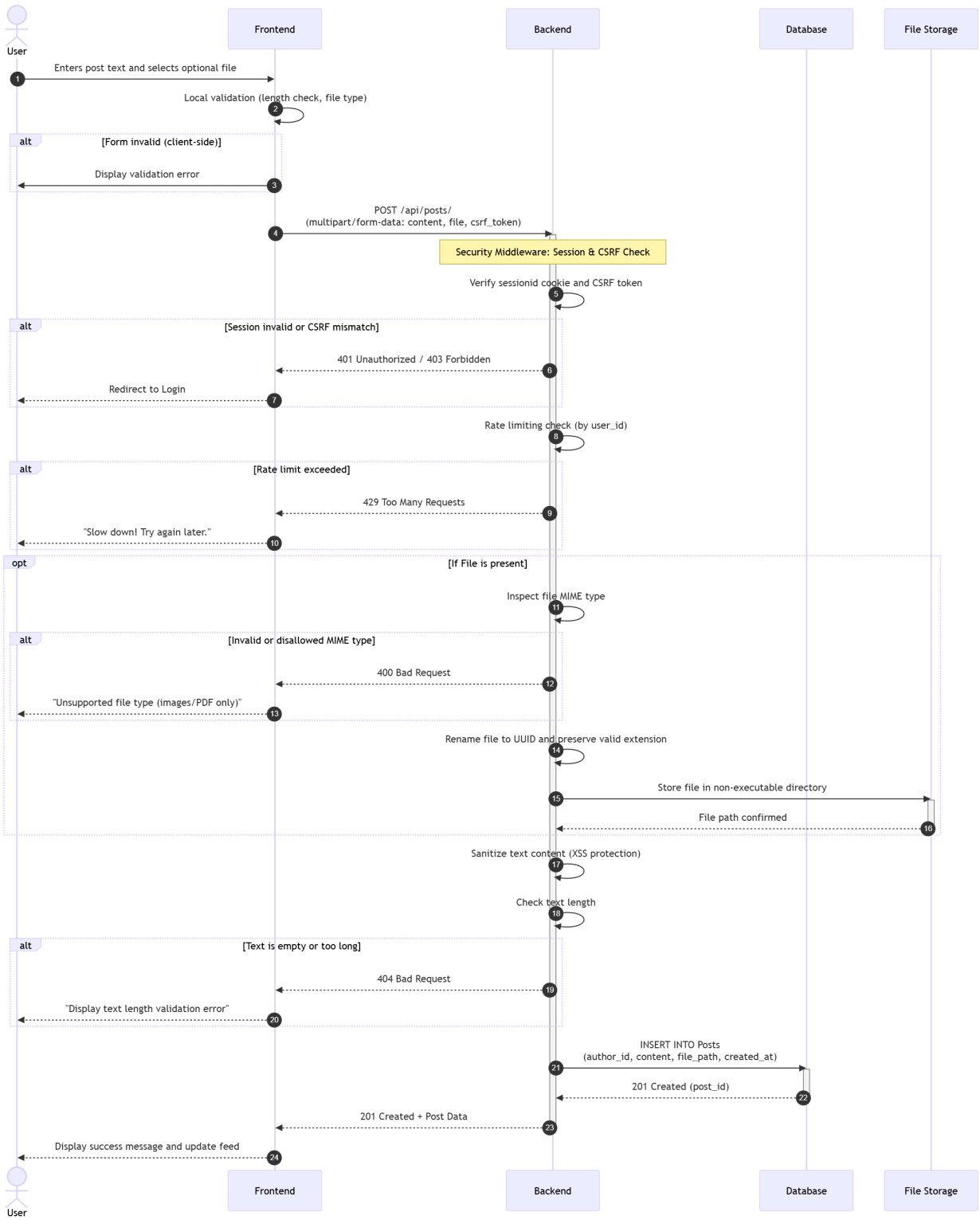


Figure 3: UML sequence diagram for creating posts

7.2. Adding comments

The UML sequence diagram for adding comments is shown in Figure 4.

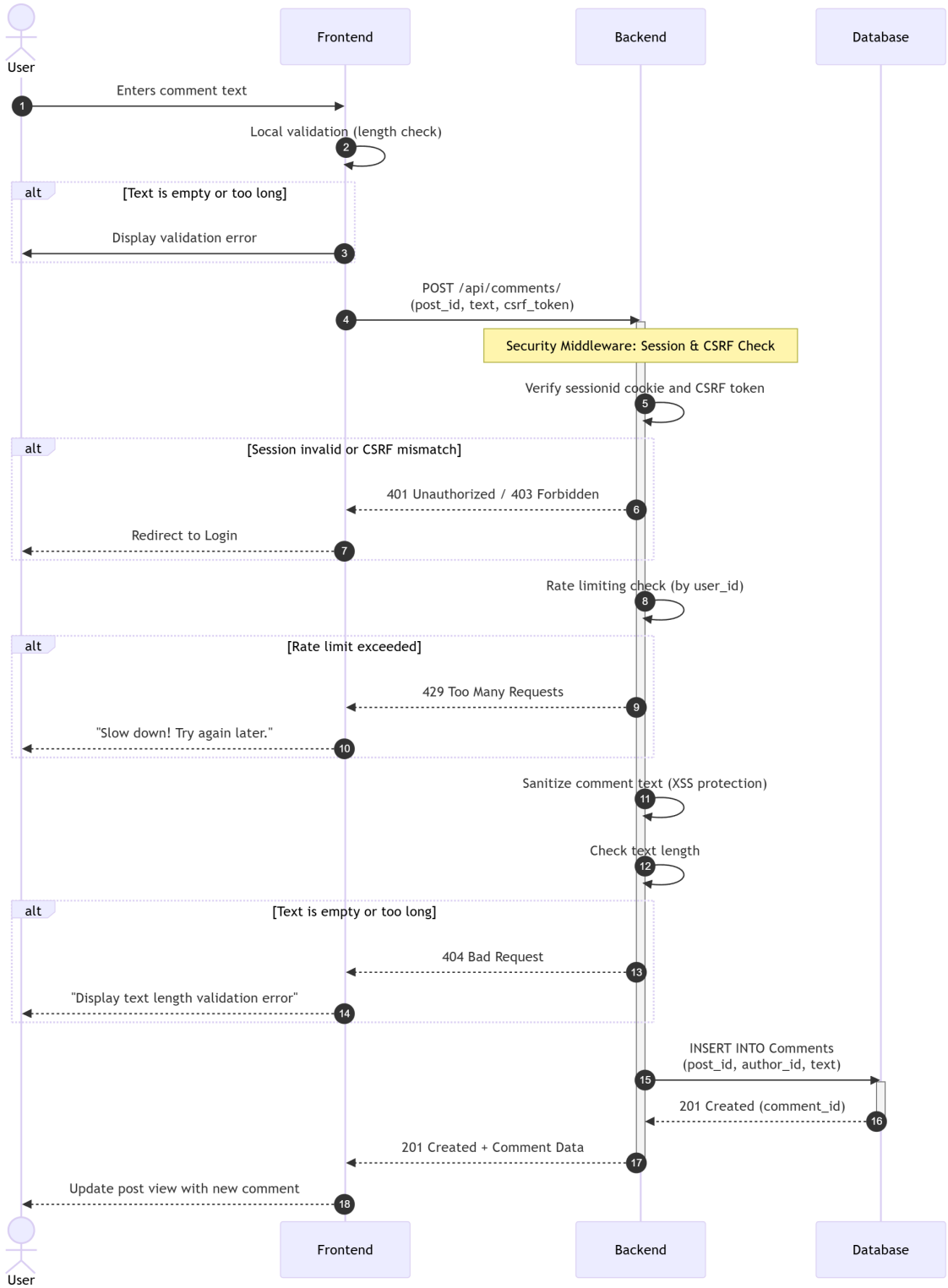


Figure 4: UML sequence diagram for adding comments

7.3. Adding ratings

The UML sequence diagram for adding ratings is shown in Figure 5.

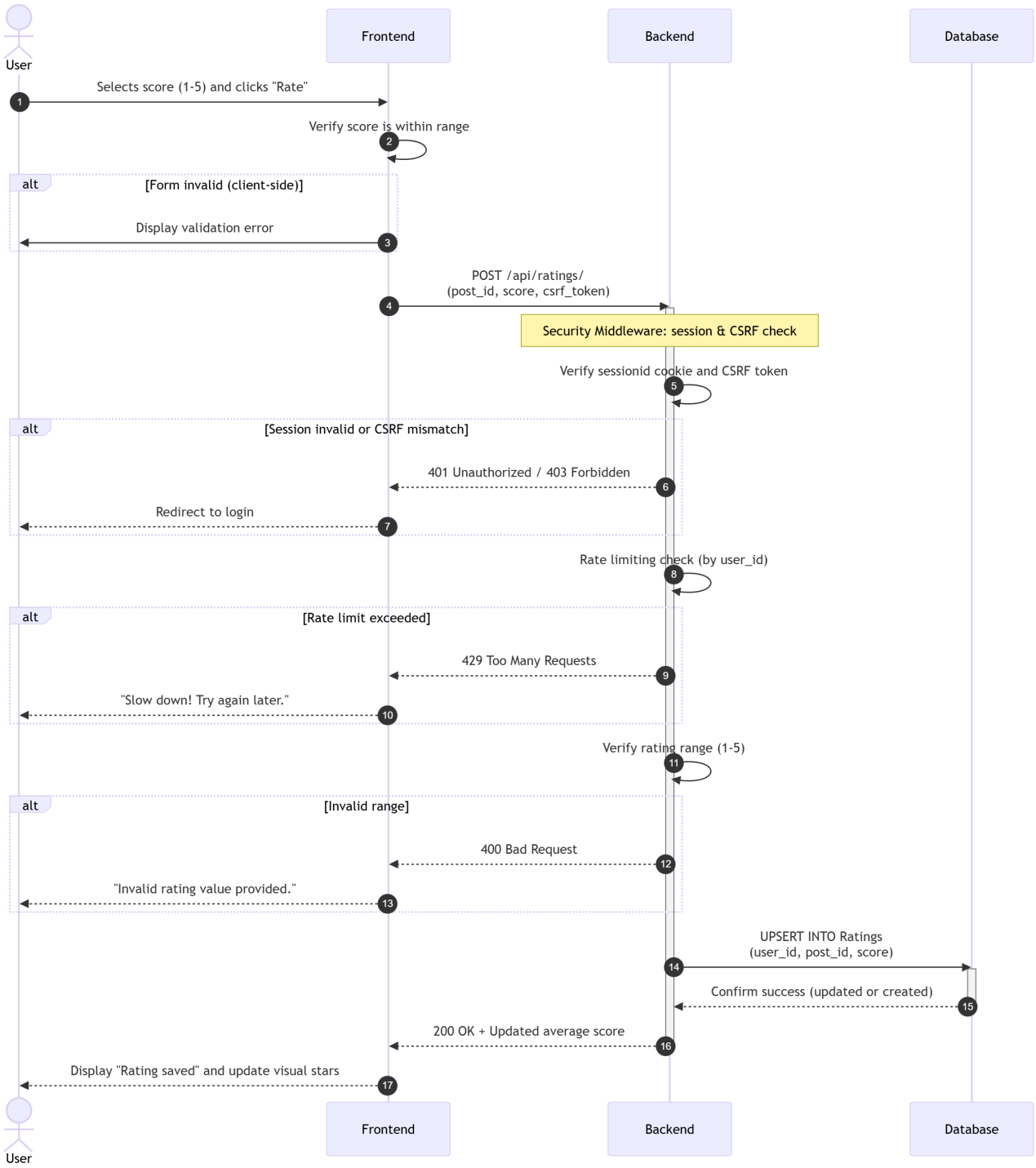


Figure 5: UML sequence diagram for adding ratings

7.4. Editing content

The UML sequence diagram for editing posts is shown in Figure 6. An analogous approach is used for editing comments, where the system verifies that the requester is either the original author or an administrator before committing changes to the database.

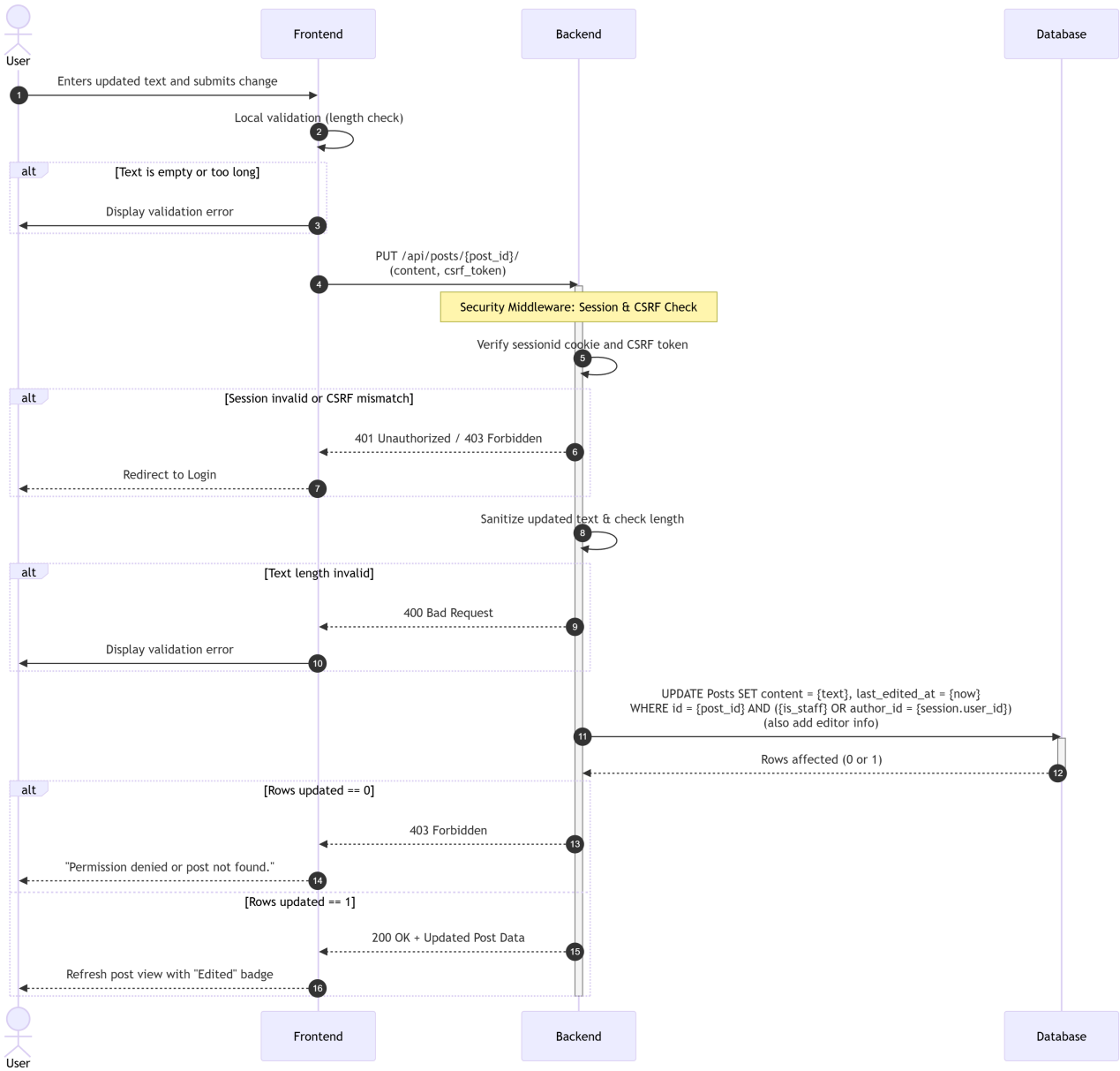


Figure 6: UML sequence diagram for editing posts

7.5. Keyword post search

The UML sequence diagram for searching posts by keywords is shown in Figure 7.

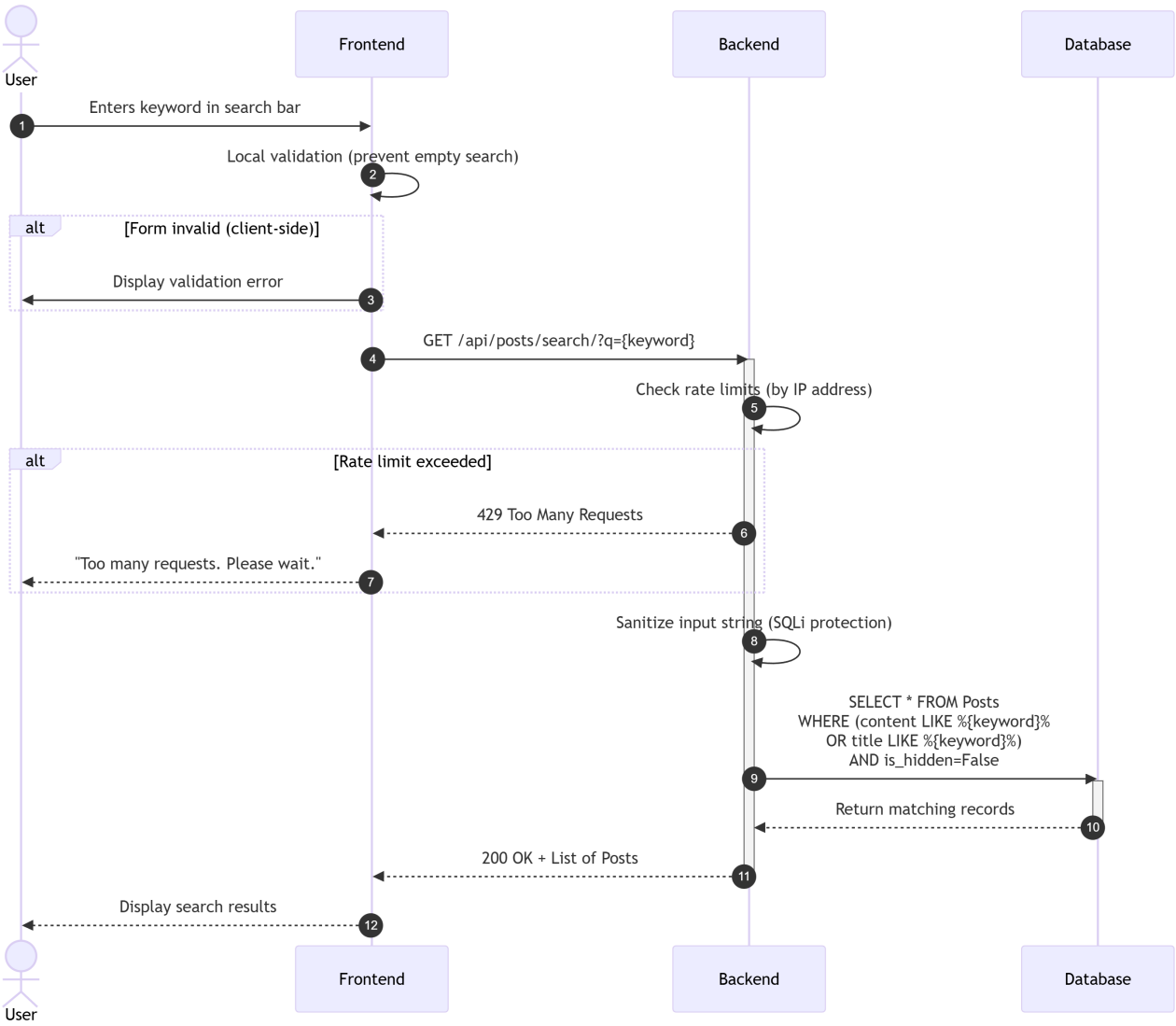
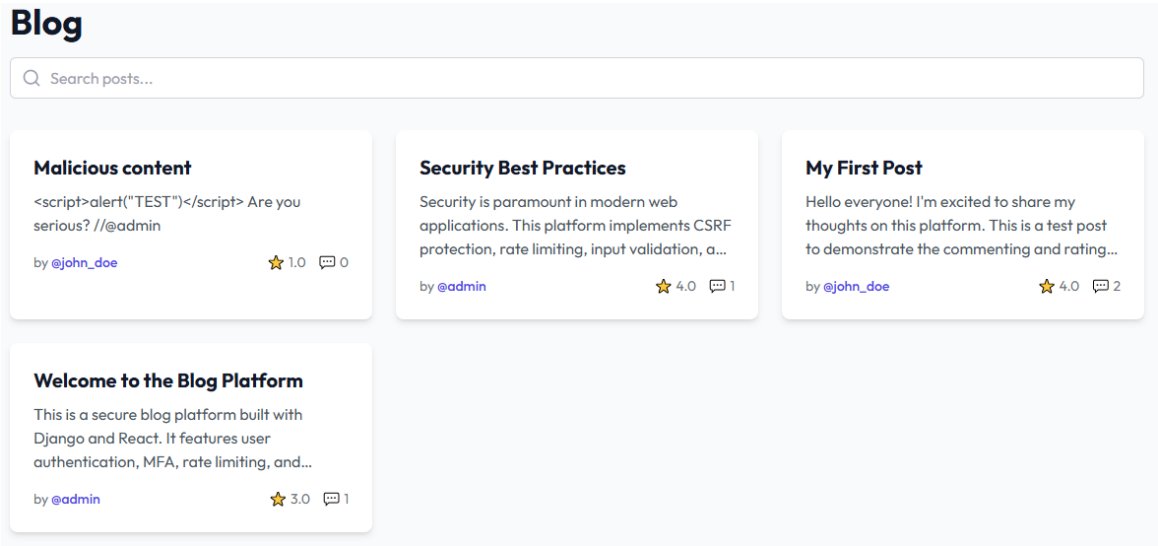


Figure 7: UML sequence diagram for searching posts by keywords

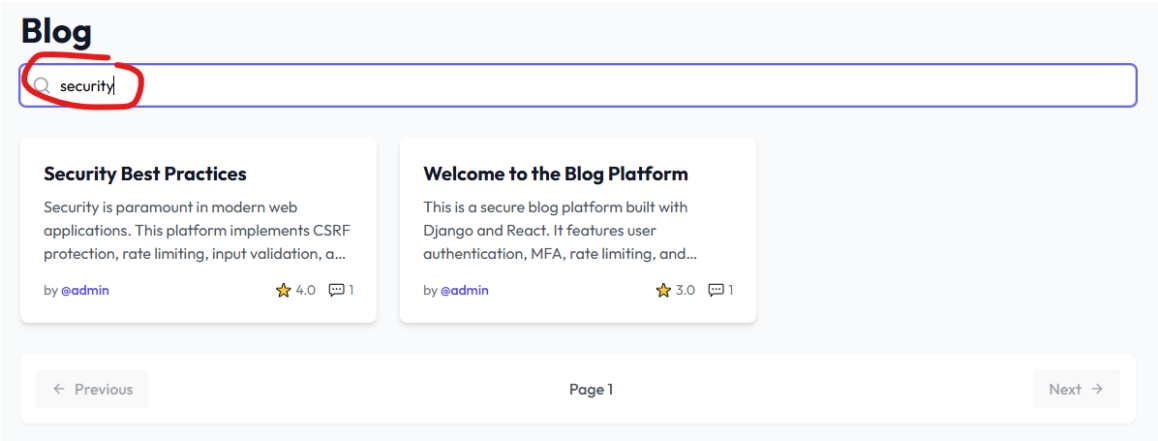
8. Implementation

8.1. Unauthenticated user view

An unauthenticated user can browse the posts through the pagination button or by searching with specific keywords, as shown in Figure 8. They cannot however add any posts.



(a) Viewing posts (home page)



(b) Filtering posts

Figure 8: Unauthenticated user view: viewing and filtering posts

Upon clicking on any post they can view its details, including comments and rating, but they cannot interact with it in any way, i.e., they can't give ratings nor add comments, as shown in Figure 9.

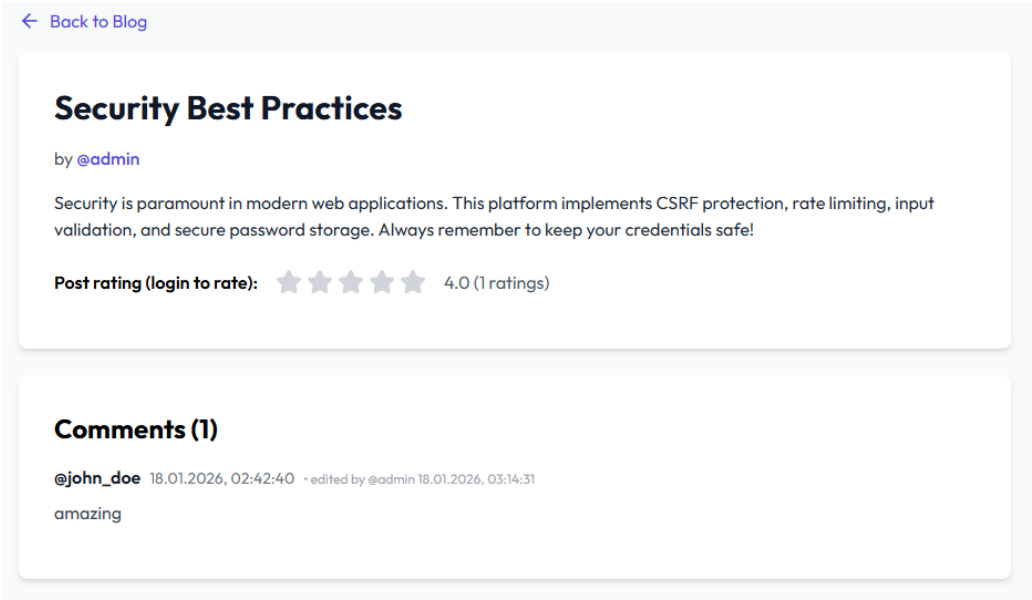


Figure 9: Unauthenticated user view: post's details – unauthenticated user can't leave ratings or comments

They can additionally view all content created by a given user by clicking on their username, as shown in Figure 10.

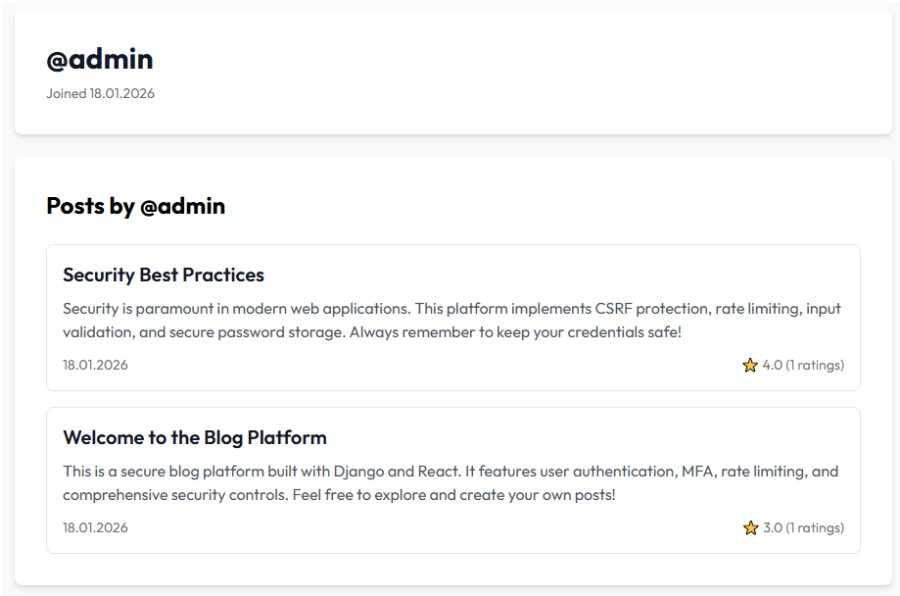
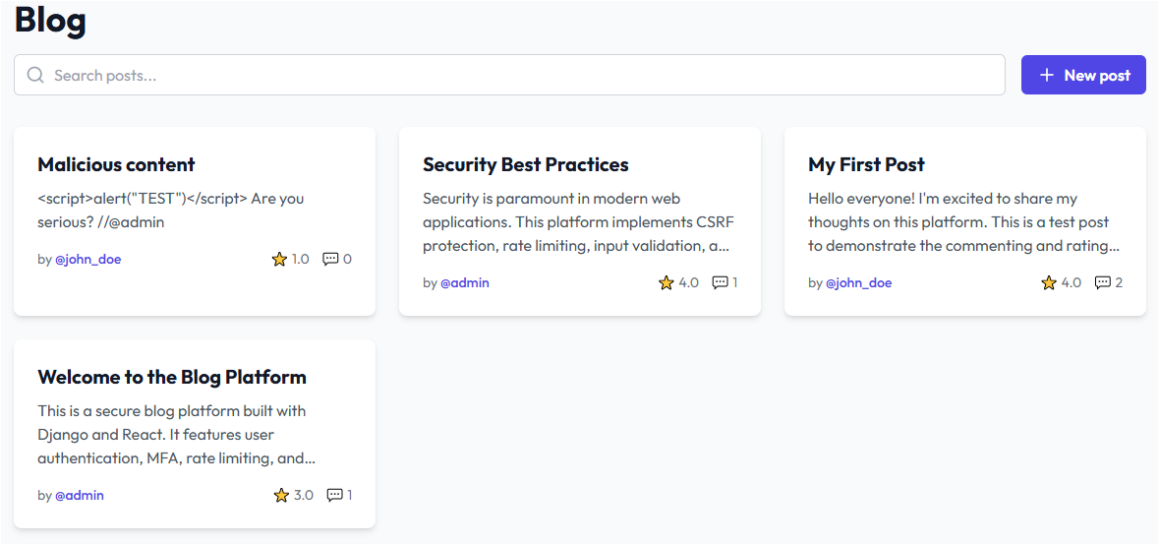


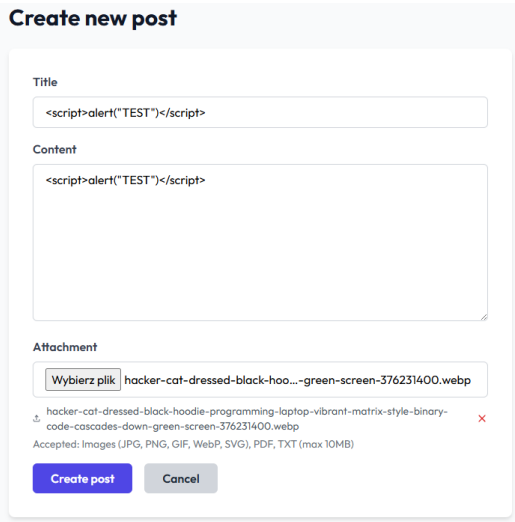
Figure 10: Unauthenticated user view: viewing user's profile with posts

8.2. Authenticated user view

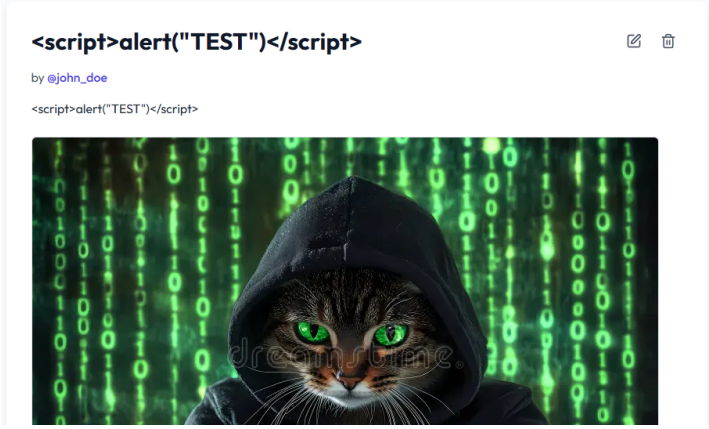
An authenticated user can perform the same read operations as an unauthenticated user. On top of that, they can create content and interact with it. After logging in, the user's home page shows the same components and an additional option to create new posts, as shown in Figure 11a. After clicking on it, the user can set the topic, add body, and select up to one attachment – either a graphic file (like png, jpg, gif) or a text document (pdf, txt), as shown in Figures 11b and 11c.



(a) Viewing posts (home page)



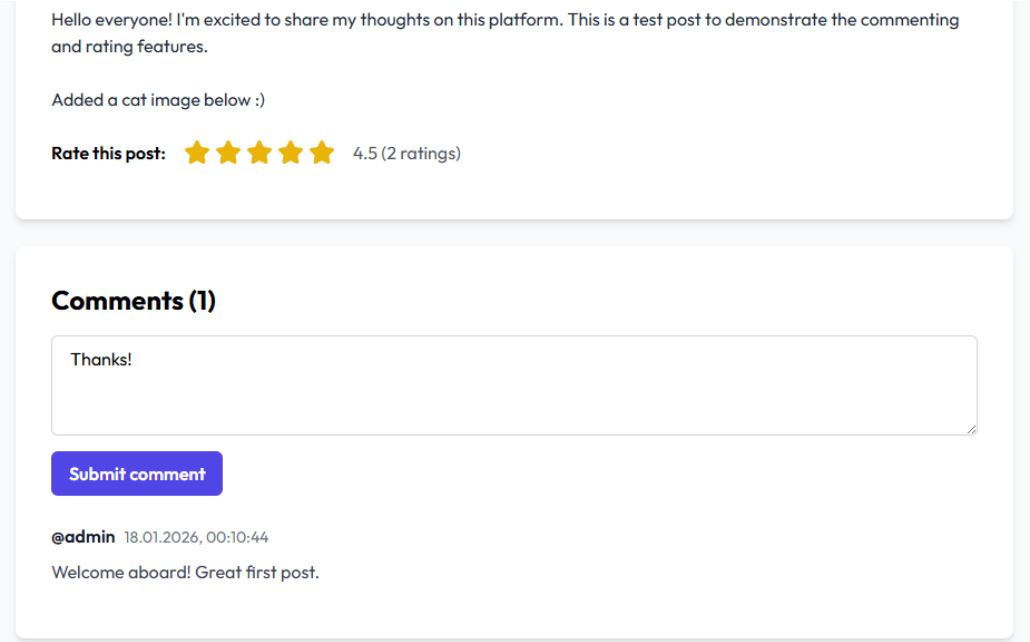
(b) Creating a post



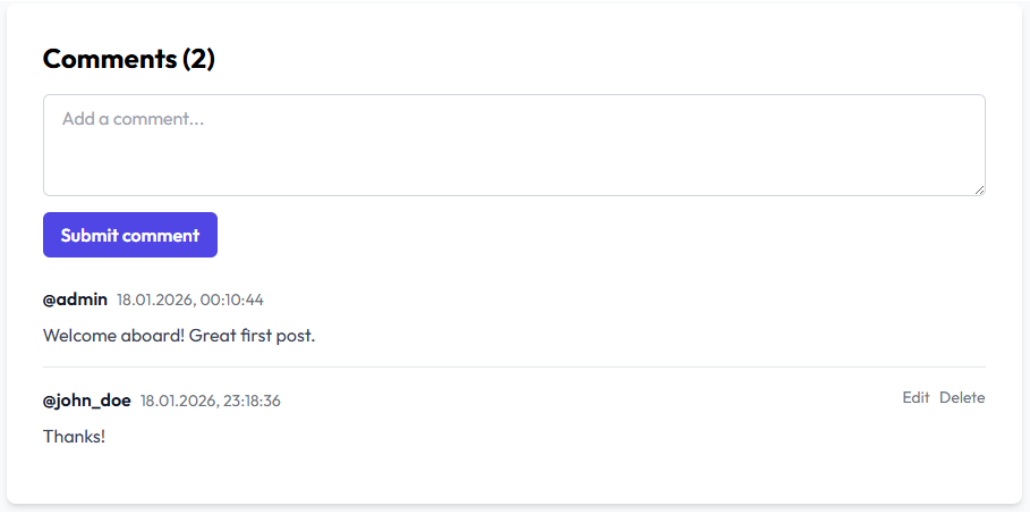
(c) Created post

Figure 11: Authenticated user view: viewing and creating posts

The user can interact with posts through star ratings or comments, as shown in Figure 12.



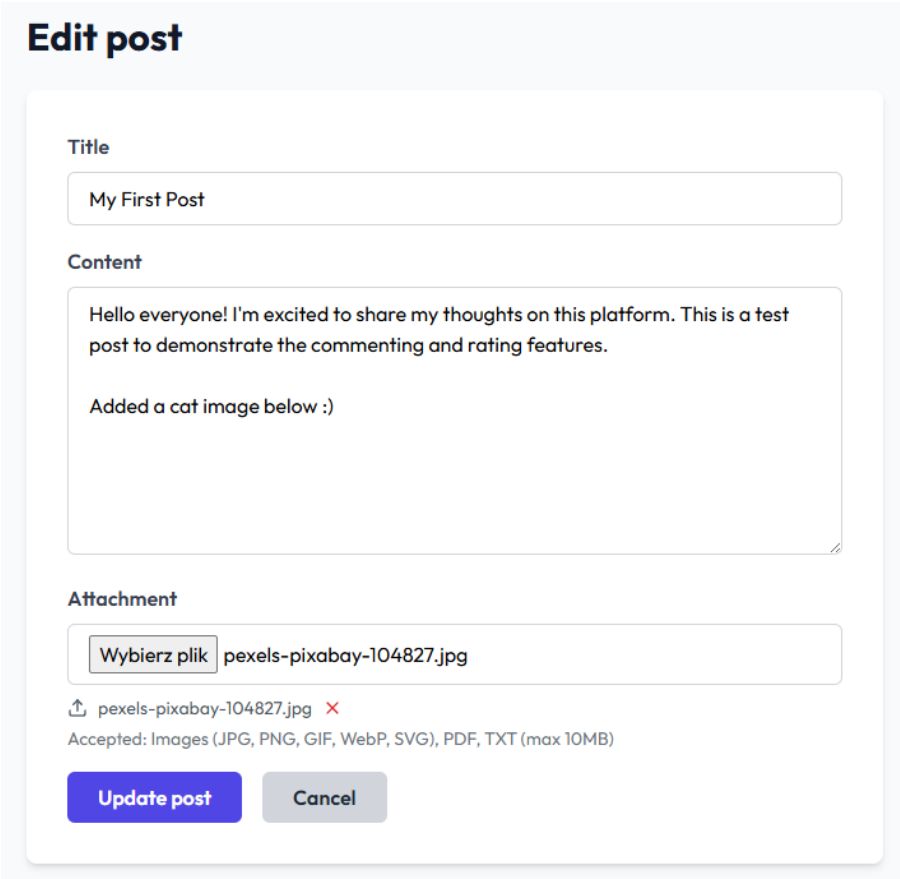
(a) Adding a comment and a rating



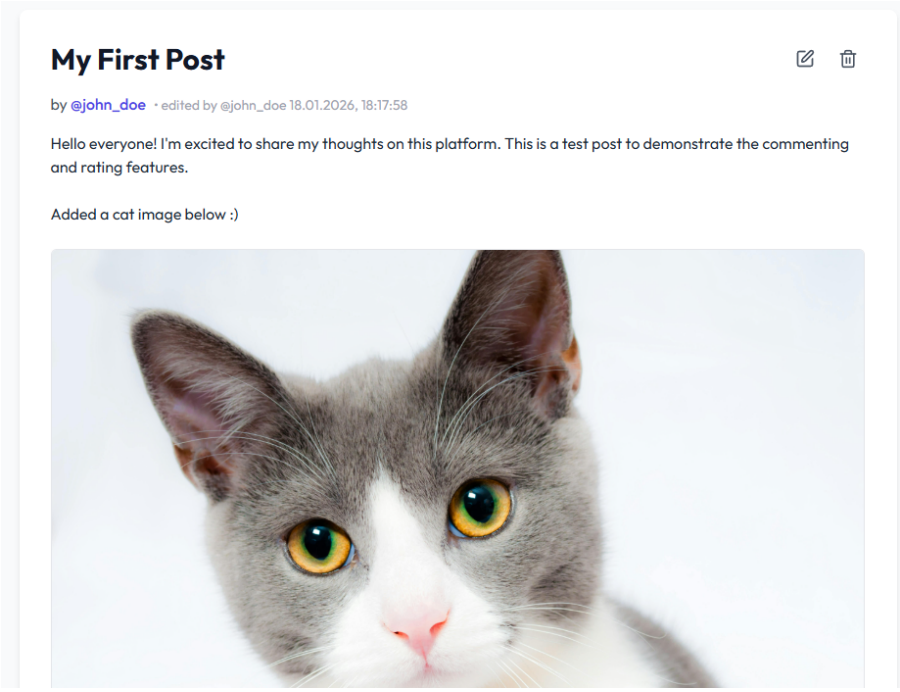
(b) Added comment

Figure 12: Authenticated user view: interacting with posts

The user can also modify the content they authored with the "Edit" functionality. They can modify post's text and change or remove the file linked in a post, as shown in Figure 13. If they want, they can also remove their content with the "Delete" button.



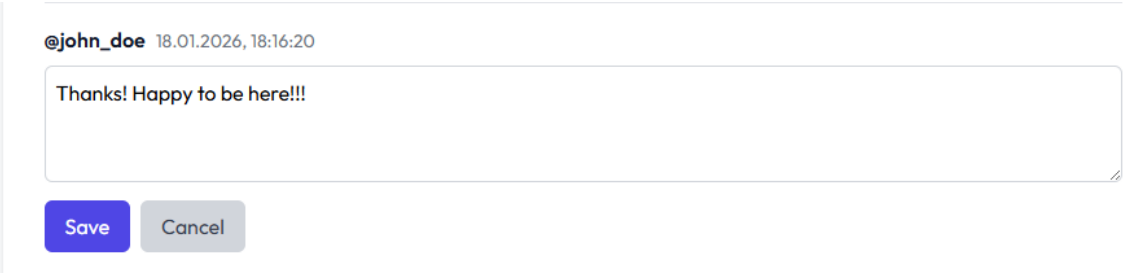
(a) Editing a post



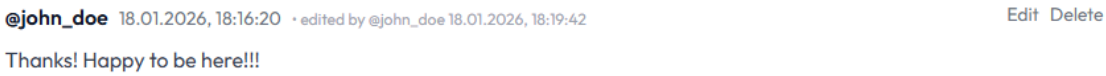
(b) Edited post

Figure 13: Authenticated user view: editing posts

The same edit and remove operations are available for comments, as shown in Figure 14.



(a) Editing a comment



(b) Edited comment

Figure 14: Authenticated user view: editing comments

By visiting their profile, an authenticated user can also see all of their posts, as shown in Figure 15.

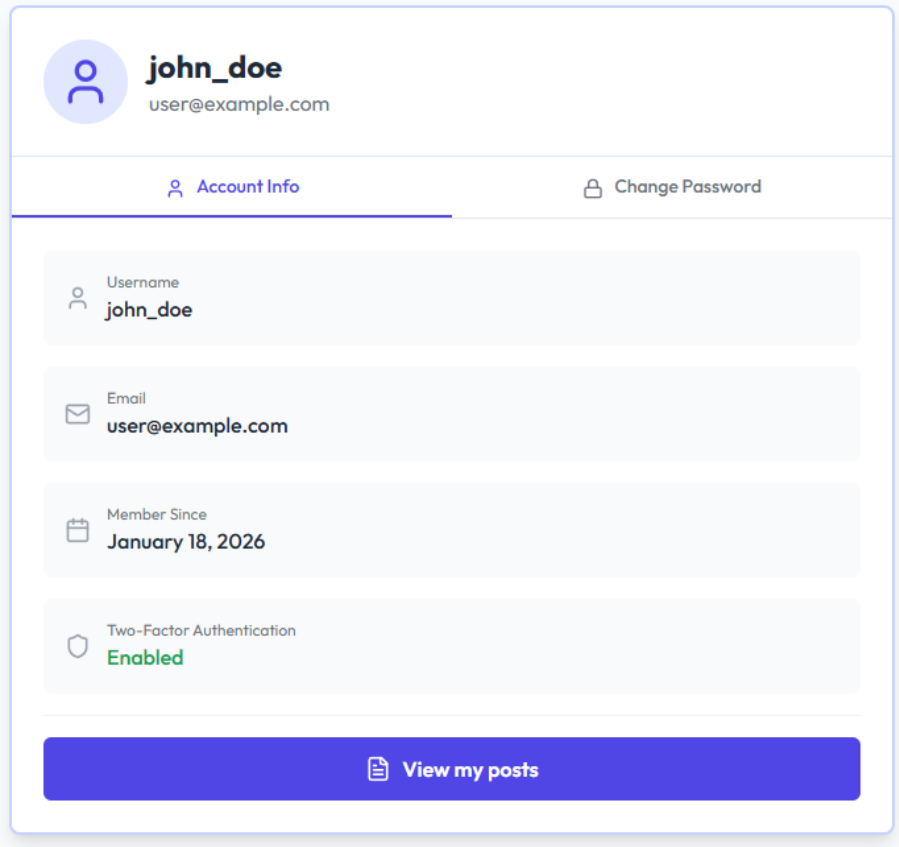
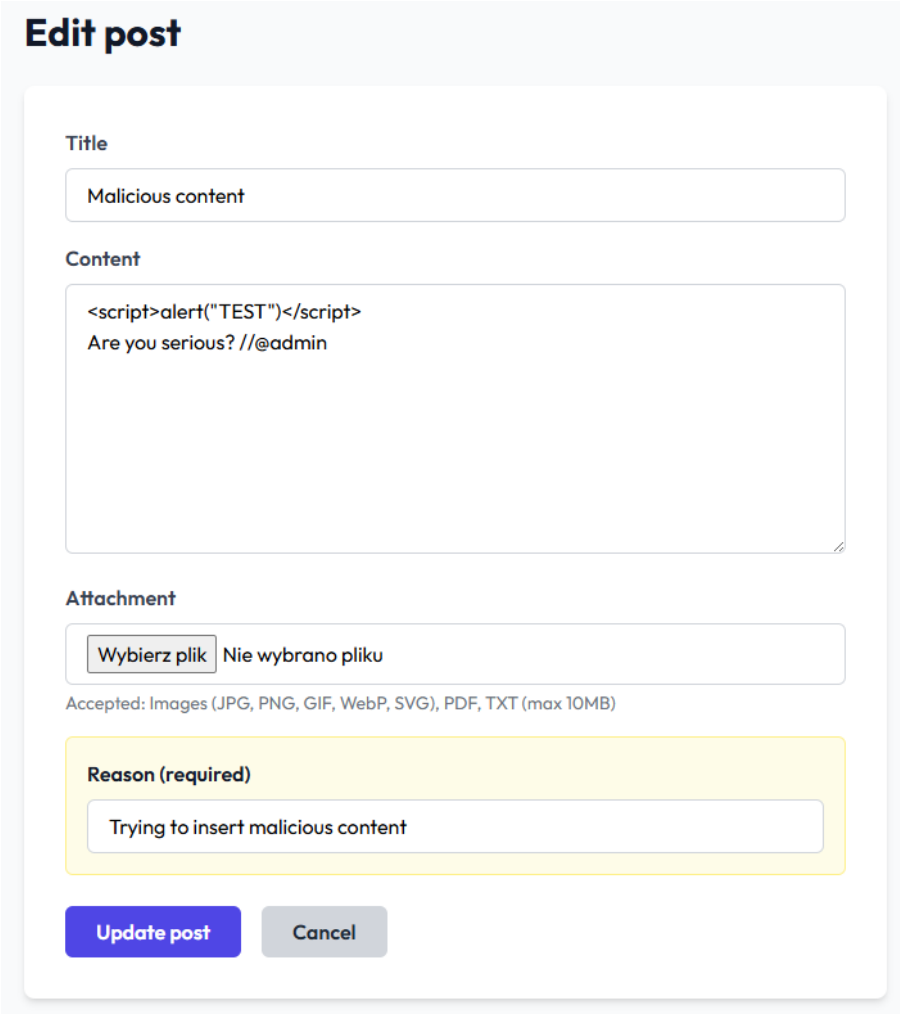


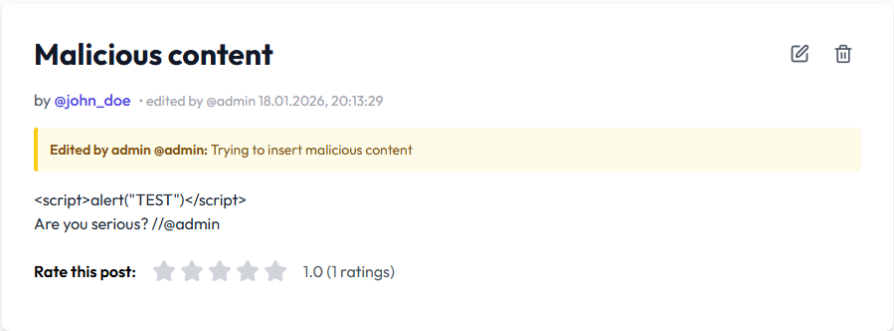
Figure 15: Authenticated user view: viewing authored posts

8.3. Admin user view

Admin users can perform the same operations as any authenticated user but on top of that they can edit or remove any content on the site. When doing so, the admin has to provide a reason for their edit (or deletion), as shown in Figure 16. The reason for editing is displayed for all admins and the content author, it is not shown to unauthenticated users but they can see the "modified by" message to indicate the last editor.



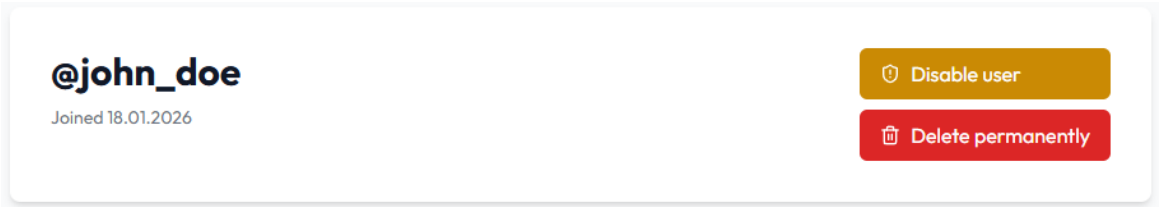
(a) Editing a post



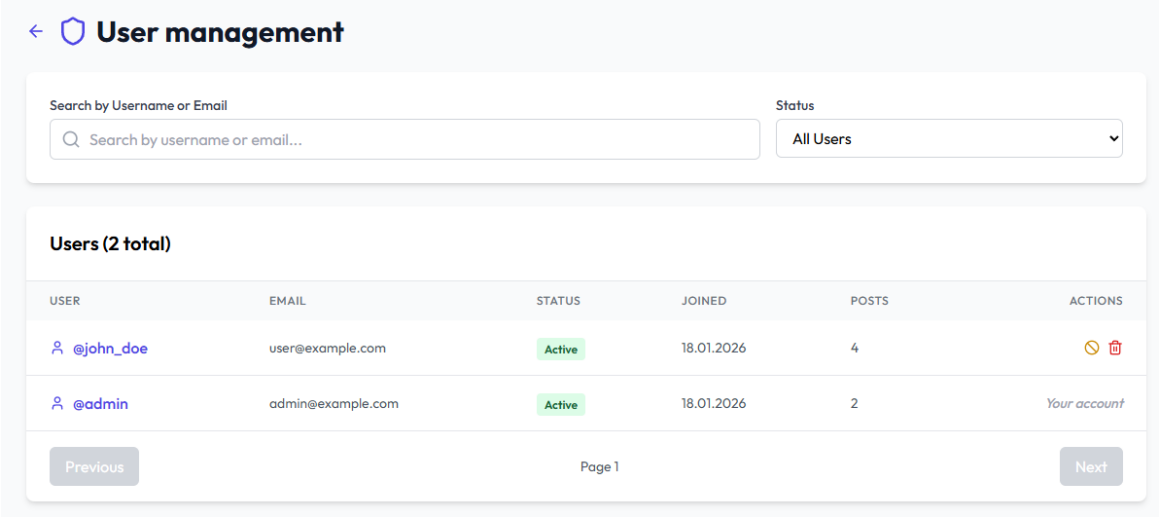
(b) Edited post

Figure 16: Admin user view: editing content

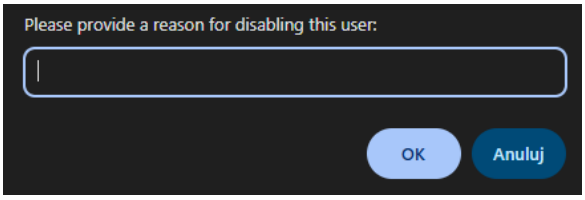
Admins can additionally manage user accounts, including the ability to disable or permanently remove them. The admin can disable a user account, which prevents the user from logging in and hides all their content (posts and comments are set to `is_hidden=True`, making them invisible to other users while preserving the data). This is a semi soft-delete feature. Alternatively, the admin can permanently delete a user, which cascades and removes all associated content from the database, including their posts, comments, and ratings. To prevent accidental self-removal, admins cannot disable or delete their own accounts through the user management interface. Admin page used for managing users, including prompts when trying to perform a specific action (either disabling or removing) are shown in Figure 17.



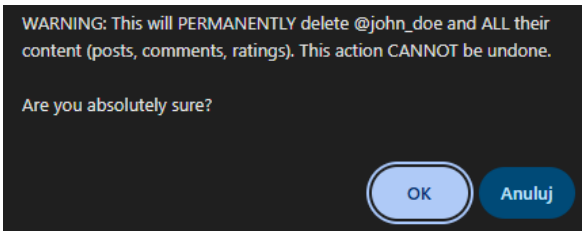
(a) User's profile



(b) User management page



(c) Reason for disabling the account



(d) Warning regarding removing an account

Figure 17: Admin user view: managing users

Management actions require the admin to provide a reason, which is logged in the AdminAction audit table. There's a dedicated page for admins that allows to browse these logs, as shown in Figure 18.

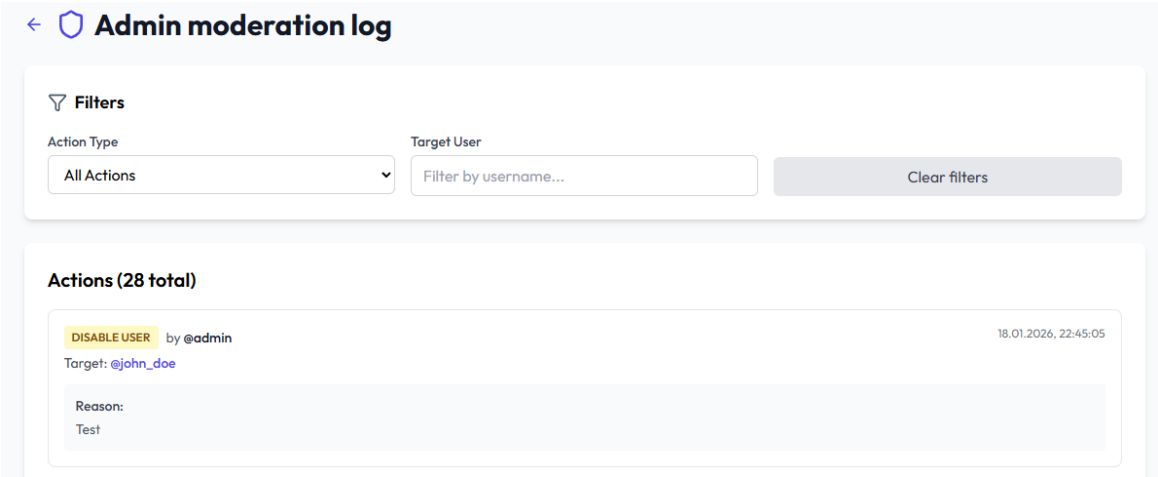


Figure 18: Admin user view: admin logs

Once user's account gets disabled, the user can try to login but gets notified that their account is disabled, as shown in Figure 19.

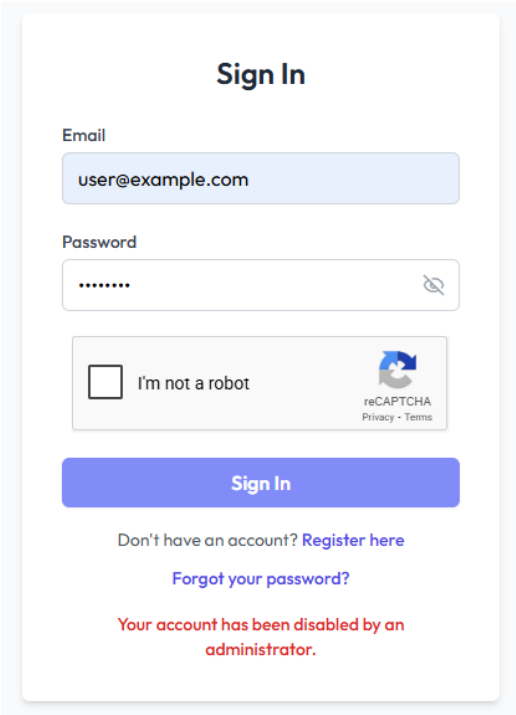


Figure 19: Login attempt after disabling an account

8.4. Details

8.4.1. Uploading files

Upon uploading a file, both file extension and MIME type validation are performed. The allowed file types include: `jpg, jpeg, png, gif, webp, svg, pdf, txt`. Maximum file size is 10 MB. MIME type validation uses the `python-magic` library to inspect actual file content, preventing users from bypassing validation by renaming files. If all checks pass, the file is saved under a random UUID-generated name in the `media/posts/` directory. This name randomization prevents path traversal and injection attacks through the filename. Once uploaded, the post object has an attachment URL in the form `media/posts/[UUID].[ext]`.

8.4.2. Security concerns

All user-provided data is sanitized through Django's ORM (preventing SQL injection) and React's automatic escaping (preventing XSS). Input validation is enforced via DRF serializers for post/comment text, emails, usernames, and file uploads.

I implemented rate limiting using Django REST Framework's throttling system to prevent abuse. I applied custom throttle classes to post, comment, and rating endpoints. Authentication endpoints use scope-based throttling configured in settings. This mitigates spam and DoS attacks while allowing legitimate usage.

Posts and comments are identified by sequential IDs, which enables enumeration. This is acceptable since all content is publicly viewable by unauthenticated users. Authorization is enforced through custom permission classes and additional view-level checks. Frontend UI elements are conditionally rendered based on user roles (`user.is_staff`), but security relies on backend enforcement, not UI hiding.

9. Conclusions

During this laboratory exercise, I designed and implemented an extension of my authentication module into a functional microblogging web service. The design covers the component's purpose, security assumptions, functional and non-functional requirements, system architecture and tech stack, database structure, and UML sequence diagrams. The features incorporated into the application include:

- role-based access control (RBAC) for Guests, Registered Users, and Administrators,
- a public feed and keyword-based search for unauthenticated visitors,
- secure post creation and editing with mandatory server-side ownership verification,
- a user-interaction system allowing for comments and numerical ratings,
- administrative moderation capabilities to manage or delete any platform content,
- other: designed and implemented as part of my authentication module.

Additional functionalities and security measures, besides the required ones, that I included in my design are:

- secure file upload handling with isolation and path randomization,
- protection against injection vulnerabilities via sanitization,
- rate limiting,
- disabling users and hiding all of their content (a semi soft-delete feature) as an administrator,
- other: designed and implemented as part of my authentication module.